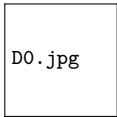


Lecture 12: Relational Algebra III & Functional Dependencies

Division, Equivalence, and The Path to Good Design

Z2004: Database Management Systems



D0.jpg

IIT Madras Zanzibar

Thursday, 26 March 2026

Recap: Joins & Trees

In Lecture 11, we expanded our Algebra toolkit:

- ▶ **Natural Join ($\bowtie$):** Matches on common columns, removes duplicates.
- ▶ **Theta Join ($\bowtie_{\theta}$):** Cartesian product followed by a specific condition (σ).
- ▶ **Outer Joins:** Keep unmatched rows.
- ▶ **Expression Trees:** Visualized how queries flow from bottom to top.

Learning Outcomes

Today, we finish Algebra and transition to Design Theory:

- ▶ **Division ($\div$):** How to answer “FOR ALL” queries.
- ▶ **Equivalence:** Why $A \bowtie B = B \bowtie A$ is the secret to fast databases.
- ▶ **The Transition:** Why do bad database designs exist?
- ▶ **Functional Dependencies (FDs):** The mathematical rules of data relationships.

1. The Division Operator ($\div$)

Answering “For All” Queries

Division is used when you need to find entities that have interacted with **every** item in a specific set.

Mathematical Notation:

*Completed_Courses $\div$
Core_Requirements*

 **Read as:** “Find students who have completed all courses listed in the Core Requirements table.”

 *Division is not a base operator! It is derived using a complex combination of Projection (π), Cross Product ($\times$), and Set Difference ($-$).*

Visualizing Division

Takes Table (A)

Student	Course
Ali	CS101
Ali	CS102
Bea	CS101
Leo	CS101
Leo	CS102
Leo	MA101

$\div$

Reqs (B)

Course
CS101
CS102



Result ($A \div B$)

Student

Ali

Leo

Bea is excluded because she only took CS101, not both!

Visualizing Division ($\div$): The Checklist Analogy

The "All or Nothing" Rule

The easiest way to understand Division is to think of the right-hand table as a **strict checklist**.

If the right-hand table has 3 items, the left-hand table must have a matching set of **all 3 items** for a specific entity to make it into the final output.

✗ Partial Matches Fail

If a student completes 9 out of 10 required courses, they do NOT graduate. Division completely drops them from the output.

The Checklist Visualized

Student DB

Name	Skill
Eve	SQL
Eve	Java
Max	SQL

$\div$

Job Req

Skill
SQL
Java



Hired

Candidates

Name	Checklist
Eve	✓✓

Max is dropped because his checklist is incomplete (✓✗).

2. Query Equivalence Rules for optimization

The Optimizer's Secret

There are many ways to write the same query. The DBMS Optimizer relies on **Equivalence Rules** to rewrite your query into the fastest possible version.

Rule 1: Commutativity of Joins

$$A \bowtie B \equiv B \bowtie A$$

Why it matters: The DBMS will always put the smaller table on the left side of the join in memory to save RAM.

Rule 2: Cascading Selections

$$\sigma_{c1 \wedge c2}(R) \equiv \sigma_{c1}(\sigma_{c2}(R))$$

Why it matters: The DBMS can split a massive WHERE clause and execute the most restrictive filter first to shrink the table size early!

Rule 3: Pushing Selections

$$\sigma_c(A \bowtie B) \equiv \sigma_c(A) \bowtie B$$

Why it matters: It is always faster to filter the tables **before** you join them, rather than joining massive tables and filtering the result.

3. Transition: Why Do We Need "Design Rules" ?

The Problem with "Flat" Data

We know how to query databases now. But what happens if the table itself is built poorly? Look at this "All-In-One" table:

Company Table

EmpID	Name	Dept	Manager
101	Alice	IT	Bob
102	Carol	IT	Bob
103	Dave	HR	Eve

If Bob is replaced by Charlie, we have to update multiple rows. **This is an Update Anomaly.**

The Three Anomalies

Bad schema design leads to data corruption through:

- ▶ **Insertion Anomaly:** We can't add a new Department (e.g., "Sales") unless we hire an employee for it first!
- ▶ **Deletion Anomaly:** If Dave quits, and we delete his row, we accidentally lose the fact that "Eve manages HR"!
- ▶ **Update Anomaly:** Data redundancy requires updating multiple places.

*To fix this, we need mathematical rules for organizing tables. Enter: **Functional Dependencies.***

4. Functional Dependencies (FDs)

What is an FD?

A Functional Dependency is a constraint that specifies a relationship between attributes.

Notation:

$$X \rightarrow Y$$

 Read as: “*X functionally determines Y.*”

Meaning:

If two rows have the exact same value for *X*, they **must** have the exact same value for *Y*.

(*X determines Y*; *Y depends on X*).

Identifying Dependencies

EmpID	SSN	Name	Dept
101	999-11	Alice	IT
102	888-22	Bob	IT
103	777-33	Alice	HR

Valid FDs in this table:

- ▶ *EmpID* $\rightarrow$ *Name* (Knowing *EmpID* guarantees knowing the *Name*)
- ▶ *SSN* $\rightarrow$ *EmpID* (*SSN* uniquely identifies *EmpID*)

Invalid FDs:

- ▶ *Name* $\nrightarrow$ *Dept* (There are two “Alice”s in different departments! *Name* does not determine *Dept*).

Visualizing FDs: The "Must Match" Rule

Valid FD: $ID \rightarrow Name$

Rule: If the *ID* is exactly the same, the *Name* **MUST** be exactly the same.

ID	Name	Role
101	Alice	Dev
102	Bob	QA
101	Alice	Lead

✓ Valid!

Every time we see $ID = 101$, the *Name* is ALWAYS "Alice". The dependency holds perfectly.

(Note: FDs don't care about the 'Role' column here, only the attributes in the rule!).

Invalid FD: $Role \rightarrow Name$

Rule Check: If the *Role* is exactly the same, is the *Name* always exactly the same?

ID	Name	Role
101	Alice	Dev
102	Bob	QA
103	Charlie	Dev

✗ Violation Found!

We see two rows where $Role = Dev$, but the names are different ("Alice" and "Charlie").

Therefore, *Role* **does not** functionally determine *Name*.

5. Attribute Closure (X^+)

Finding the Closure

The **Closure** of a set of attributes X (written as X^+) is the set of ALL attributes that can be functionally determined by X .

Why do we care?

If the closure of an attribute gives you **every single column** in the table, then that attribute is a **Superkey**!

The Algorithm:

1. Start with $X^+ = X$.
2. Check given FDs. If the left side is entirely inside your closure, add the right side to your closure.
3. Repeat until nothing new is added.

Closure Example

Let Table $R = (A, B, C, D)$.

Given FDs: $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

Find the Closure of A (A^+):

- ▶ Start: $A^+ = \{A\}$
- ▶ Because $A \rightarrow B$, add B :
 $A^+ = \{A, B\}$
- ▶ Because $B \rightarrow C$, add C :
 $A^+ = \{A, B, C\}$
- ▶ Because $C \rightarrow D$, add D :
 $A^+ = \{A, B, C, D\}$

Conclusion:

Since A^+ contains all attributes in the table, **A is a Superkey** for R!

Class Activity: FDs & Closures

The Data & Rules

Database Table: Projects

P_ID	Manager	Budget
P1	Smith	10k
P2	Jones	20k
P1	Smith	10k
P3	Smith	50k

A New System's Business Rules (FDs): Let Table $R = (A, B, C, D, E)$

Given the following FD set F :

- ▶ $A \rightarrow B$
- ▶ $B \rightarrow C$
- ▶ $A, C \rightarrow D$
- ▶ $D \rightarrow E$



Instructions:

Take 5 minutes to solve the 2 tasks on the right.

The Tasks

Task 1: Spot the Violation

- ▶ Look at the **Projects** table on the left.
- ▶ Does the dependency $Manager \rightarrow Budget$ hold true?
- ▶ *Briefly explain why or why not.*

Task 2: Compute the Closure (X^+)

- ▶ Look at the Business Rules for Table R .
- ▶ Compute the closure of A . (Find A^+).
- ▶ Show your work step-by-step.
- ▶ **Bonus Question:** Is attribute A a Superkey for table R ?

Class Activity: Task 1 Solution

Does *Manager* $\rightarrow$ *Budget* hold true?

Answer: NO.

P_ID	Manager	Budget
P1	Smith	10k
P2	Jones	20k
P1	Smith	10k
P3	Smith	50k

Explanation:

We have two rows where the Manager is “Smith”. In one row, the Budget is 10k. In the other row, the Budget is 50k.

Because the same Manager points to **two different** Budgets, the Functional Dependency is violated!

Class Activity: Task 2 Solution

Compute A^+

Given the rules: $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D, D \rightarrow E\}$

Step-by-Step:

1. **Start with the base attribute:**

$$A^+ = \{A\}$$

2. **Apply $A \rightarrow B$:**

$$A^+ = \{A, B\}$$

3. **Apply $B \rightarrow C$:**

$$A^+ = \{A, B, C\}$$

4. **Apply $A, C \rightarrow D$ (We have both A and C in our closure, so we can add D !):**

$$A^+ = \{A, B, C, D\}$$

5. **Apply $D \rightarrow E$:**

$$A^+ = \{A, B, C, D, E\}$$

Bonus Answer: YES. Because A^+ contains every attribute in the table R , A is a Superkey!

Summary & Next Lecture

Key Takeaways

- ✓ **Division ($\div$):** Used to query “for all” conditions.
- ✓ **Equivalence:** Database optimizers use rules like commutativity and cascading selections to speed up queries.
- ✓ **Anomalies:** Flat, poorly designed tables cause Insert, Update, and Delete anomalies.
- ✓ **Functional Dependencies ($X \rightarrow Y$):** If X is the same, Y must be the same.
- ✓ **Closure (X^+):** The algorithm to find all attributes determined by X, used to find Keys!

Next Lecture (L13)

Tuple Relational Calculus (TRC)

- ➔ Declarative vs. Procedural query logic (Algebra vs. Calculus).
- ➔ TRC Syntax: Variables, logic gates ($\wedge, \vee, \neg$), and quantifiers ($\exists, \forall$).
- ➔ Writing practical TRC expressions and mapping them to SQL.

Reading Assignment

- 📖 Read **Silberschatz, Chapter 6** (Sections on Tuple Relational Calculus) to prepare.